

ADAPTIVE AI LEARNING SYSTEM

V1.2 Agentic Architecture & Implementation Plan

No RAG in V1 • Stateful LangGraph • Persistent Learner Model • Adaptive Teaching Loop

Purpose: build a real agentic learning product that learns how each user learns, maintains uncertain knowledge state, chooses actions, adapts teaching strategy, and stays fast enough for interactive use.

Recommended stack: Android + FastAPI + LangGraph + LangChain + PostgreSQL + LangSmith

1. Product Thesis

The product is not “an LLM with a better prompt.” The product is a stateful adaptive learning system that repeatedly observes the learner, decides what action is useful, teaches, gathers evidence, updates its belief about the learner, and changes future behavior.

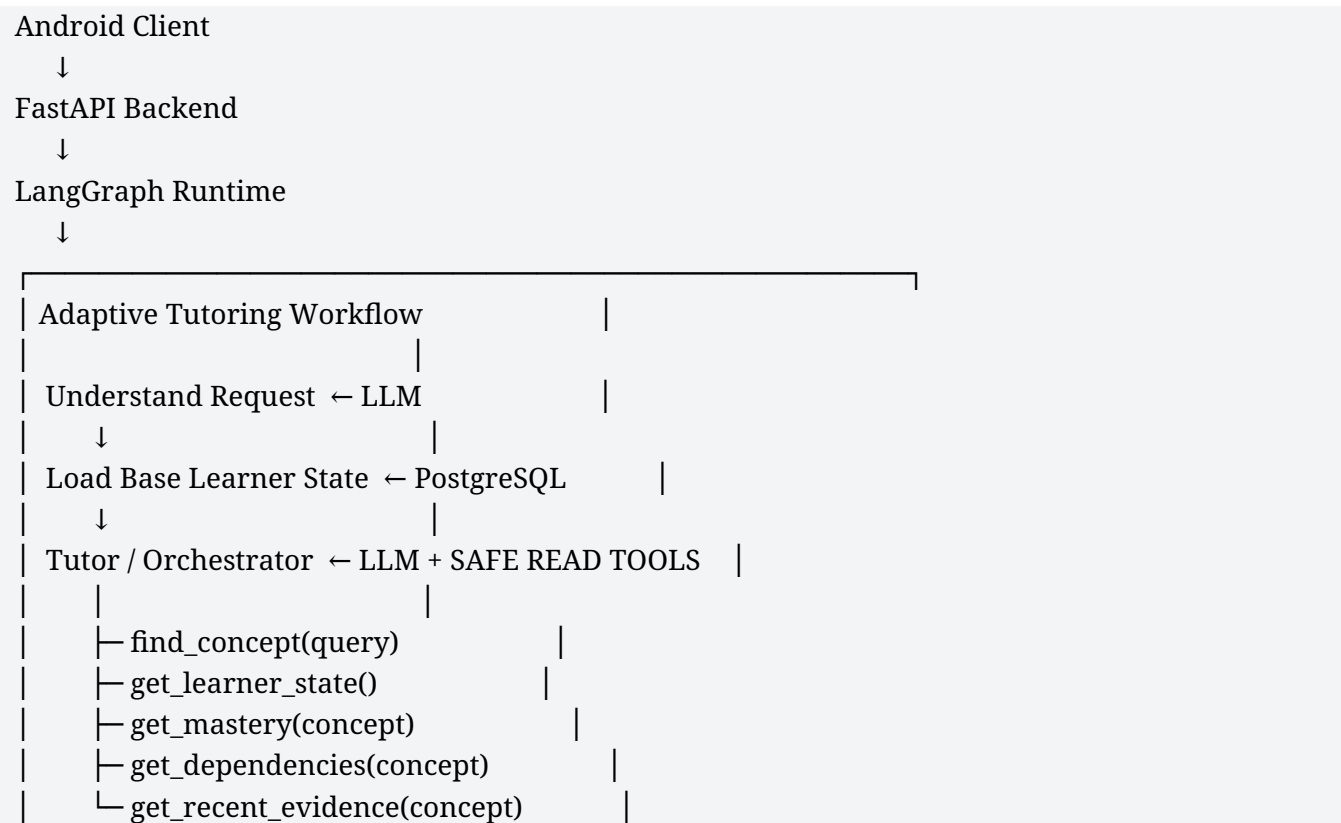


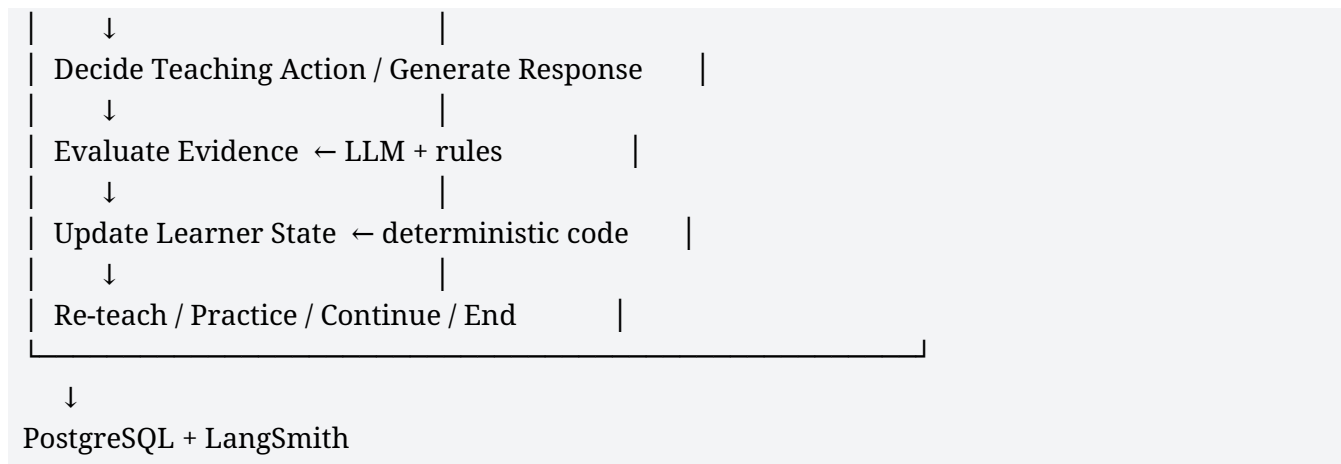
V1 focuses on one tutoring agent implemented as a LangGraph workflow. It may call the same underlying LLM several times for different roles, but the workflow itself contains deterministic nodes, database operations, branching, loops, and state updates.

2. V1 Scope

- One overall adaptive tutoring agent.
- One LangGraph workflow.
- One primary LLM model/provider is enough initially.
- Multiple LLM-backed roles: request understanding, tutor, evaluator, optional planner.
- Persistent learner model: knowledge state + learning behavior/profile.
- Sparse, demand-driven concept registry and dependency graph.
- Deterministic mastery/confidence updates and routing.
- Fast synchronous path: usually 2 LLM calls for a normal teaching turn.
- No RAG, no vector database, no multi-agent swarm in V1.

3. Core Architecture

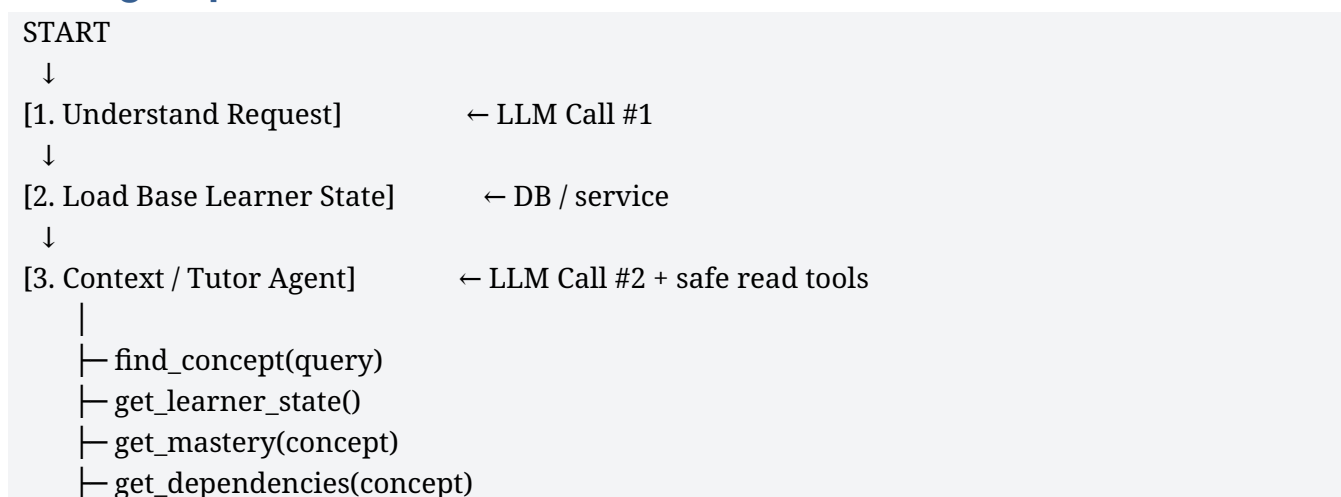


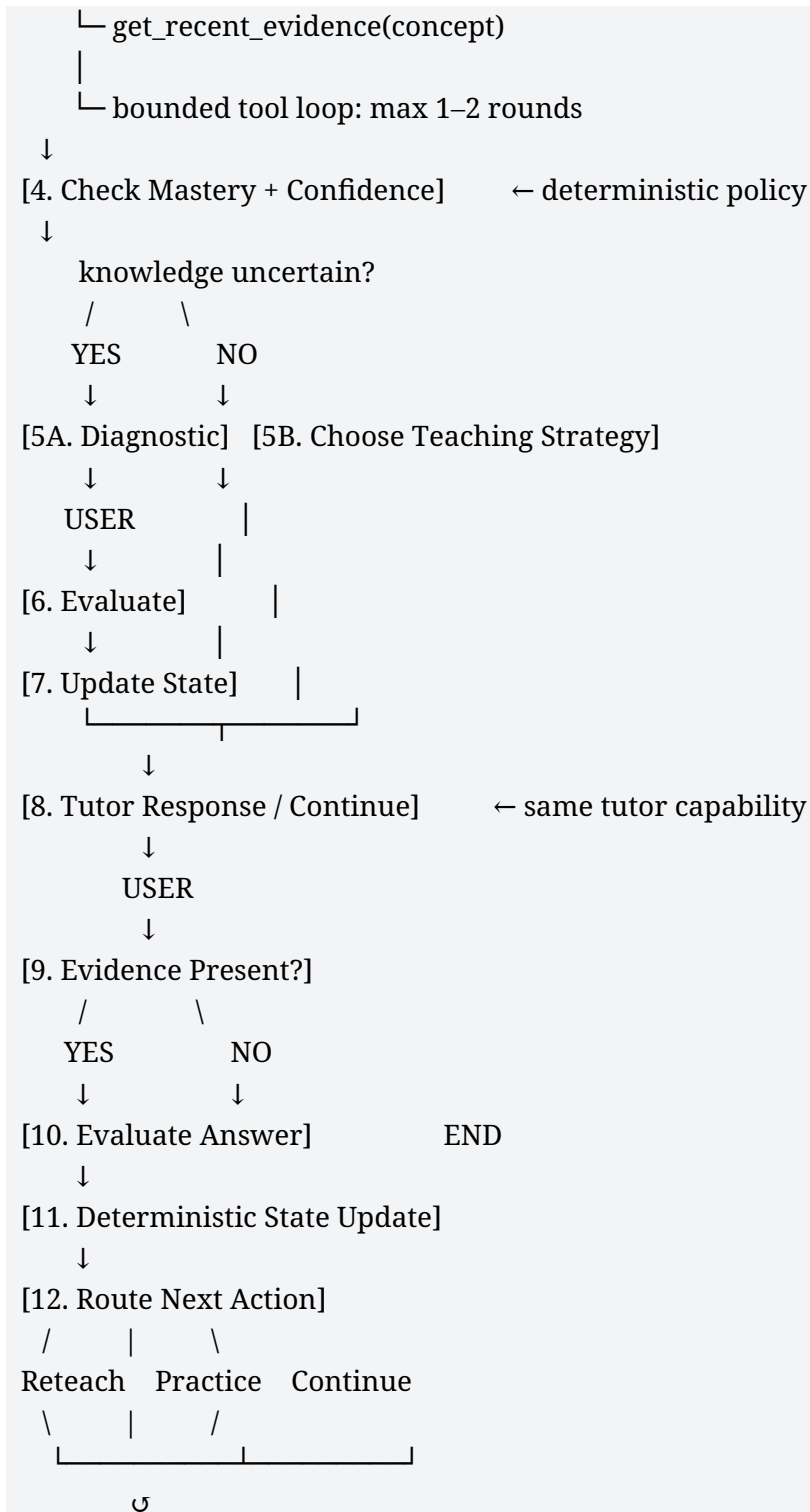


4. What Makes It Agentic

Capability	V1 behavior
Observe	Reads session state, learner profile, concept mastery, confidence, and recent evidence.
Decide	Chooses whether to teach, diagnose, re-teach, practice, or continue.
Act	Generates a lesson, asks a diagnostic, creates an exercise, or invokes a state/tool operation.
Evaluate	Scores or interprets the learner’s response using structured output plus deterministic checks.
Update	Changes mastery, confidence, misconceptions, and learning-preference signals.
Loop	Routes to the next action based on updated state rather than ending after one generation.

5. LangGraph Workflow





6. LLM Invocation Strategy

Normal teaching turn: target 2 LLM stages. The second stage may perform a bounded safe tool-calling loop before answering.

Call	Purpose	Tools?
#1	Understand intent, target	No tools. Keep this call narrow

	concept, and interaction type; return structured output.	and fast.
Between calls	Load only minimal base/session state needed to start the turn.	Deterministic DB/service calls.
#2	Tutor/orchestrator resolves the canonical concept when needed, acquires only useful learner/context data, then generates the personalized response.	YES: allow-listed tools such as find_concept, get_mastery, and get_dependencies; cap at 1-2 tool rounds.
Optional #3	Evaluate a learner answer when an assessment/diagnostic actually occurs.	No authoritative DB writes. Return structured evidence; code validates and persists.

The same underlying LLM model can serve request understanding, tool-enabled tutoring/orchestration, and evaluation. The roles stay separated by prompts, schemas, tools, and graph nodes.

6.1 Controlled Tool-Calling Policy

Rule: the LLM may decide what to look up, call allow-listed tools, interpret returned candidates, and propose actions. It does not directly perform unrestricted authoritative database mutations.

Validated application code owns persistence and invariants.

Safe V1 tools: find_concept(query), get_learner_state(), get_mastery(concept), get_dependencies(concept), get_recent_evidence(concept). The LLM may choose these tools to resolve meaning and fetch context. Avoid broad web, shell, arbitrary SQL, filesystem, or unrestricted write tools.

7. Learner Model

The learner model is the core product asset. It should never claim certainty where none exists.

State	Example fields
Knowledge state	concept_id, mastery_estimate, confidence, evidence_count, last_evidence_at, misconception_tags
Learning profile	top_down, example_first, causal_reasoning, visual_structure, code_preference, preferred_depth, pacing
Behavior signals	asks_for_why, asks_for_examples, repeated_confusion_patterns, successful_explanation_strategies
Session state	current_concept, current_action, recent_messages, pending_diagnostic, retry_metadata

Important rule: no UserConceptState record means unknown, not zero. Old mastery should reduce confidence in the estimate rather than automatically assuming the learner forgot everything.

8. Sparse Concept and Dependency Model

Do not pre-build a graph of all human knowledge. Create concepts lazily when they become relevant.

User asks: "Teach me transformers"

↓

Resolve:

- Transformers
- Attention
- Neural Networks
- Vectors

↓

Reuse existing concepts where possible

Create only missing concepts

↓

Store only local prerequisite edges needed by this learning path

The dependency graph answers: what prerequisite is blocking this user right now? It should not recursively traverse forever. V1 should cap depth and return only the few prerequisites that materially affect the current task.

8.1 Agent-Controlled Concept Resolution

When the user writes a free-form request, the Understand Request LLM extracts the likely target topic. The Tutor/Orchestrator can then call `find_concept(query)` to search the concept registry.

`find_concept` is a deterministic tool backed by the concept service/database. It searches exact names and aliases first, then can use fuzzy text matching to return a small candidate list. The LLM uses the original conversation context to choose the intended candidate when the result is ambiguous.

If no candidate is confident enough, the LLM proposes a normalized new concept. Deterministic code validates, de-duplicates, assigns the canonical `concept_id`, and persists it. The LLM can then call `get_mastery()` and `get_dependencies()` for the resolved concept.

User message -> LLM extracts topic -> `find_concept(topic)` -> candidate concepts -> LLM resolves ambiguity -> validated canonical concept -> mastery/dependency tools

9. Data Model

Entity	Purpose
users	User account reference and basic product

	metadata.
learner_profiles	Long-term learning-behavior estimates and confidence.
concepts	Canonical concept registry; sparse and demand-driven.
concept_edges	Prerequisite / related concept relationships.
user_concept_state	Mastery estimate, confidence, recency, evidence count.
learning_evidence	Quiz/diagnostic/interaction evidence used to update state.
sessions	Persistent learning session and current workflow status.
interaction_signals	Signals such as 'needs example', 'too detailed', or repeated successful strategy.

10. Technology Stack

Layer	V1 choice
Android	Kotlin + Jetpack Compose + ViewModel + StateFlow
Local Android persistence	Room
Backend	Python + FastAPI + Pydantic
Agent orchestration	LangGraph
LLM integration	LangChain model abstractions + structured outputs
Primary database	PostgreSQL
ORM/migrations	SQLAlchemy + Alembic
Observability/evaluation	LangSmith
Deployment	Dockerized backend + managed PostgreSQL
RAG	Not included in V1

11. Backend Code Boundaries

```

app/
  api/
    routes/
    schemas/

ai/

```

```
graph/  
nodes/  
prompts/  
tools/      # allow-listed LLM tool wrappers  
  concept_tools.py  # find_concept + concept context reads  
llm_schemas/  
  
domain/  
  learner/  
  mastery/  
  concepts/  
  routing/  
  
services/  
  learner_service/  
  concept_service/  
  evidence_service/  
  
infrastructure/  
  db/  
  repositories/  
  llm/  
  observability/  
  
tests/
```

The domain layer should be testable without making an LLM call. LangGraph orchestrates; it should not own your business rules.

12. Phased Implementation Plan

Phase 0 — Architecture skeleton

Build: Create the repository structure, GraphState schema, core domain interfaces, and empty service boundaries. No production behavior yet.

Exit criterion: You can add future nodes without reorganizing the entire project.

Phase 1 — Minimal stateful tutoring loop

Build: Implement Understand Request → Load Learner State → Tutor Response. Use a hardcoded learner profile first.

Exit criterion: A question can travel through LangGraph and the learner state visibly changes the final explanation.

Phase 1B — Controlled tool-calling extension

Build: Keep the Phase 1 graph intact. Make the Tutor/Orchestrator node tool-enabled with a tiny allow-list of read-only tools: `get_learner_state`, `get_mastery`, `get_dependencies`, and `get_recent_evidence`. Cap the tool loop at 1–2 rounds.

Exit criterion: The LLM can choose when it needs extra learner context instead of receiving every possible field up front, while authoritative state writes remain deterministic.

Phase 2 — PostgreSQL learner state

Build: Replace hardcoded state with real `learner_profile`, `concepts`, `user_concept_state`, `sessions`, and `evidence` tables.

Exit criterion: State survives process restarts; old Phase 1 nodes continue working through service interfaces.

Phase 3 — Concept resolver + sparse dependencies

Build: Add agent-controlled concept resolution. The LLM extracts the target topic and may call `find_concept(query)`. The tool searches the registry and returns candidates; the LLM resolves ambiguity from conversation context. If no confident match exists, the LLM proposes a new normalized concept and deterministic code validates/de-duplicates/persists it. Add local prerequisite edges and keep unknown concepts lazy.

Exit criterion: Messy or ambiguous topic names resolve to one canonical concept when possible, new concepts are safely created when necessary, and the system can fetch a small relevant prerequisite neighborhood without an infinite graph.

Phase 4 — Diagnostic and uncertainty

Build: Add mastery confidence, evidence freshness, and the branch that runs a short diagnostic when knowledge is unknown/stale.

Exit criterion: The system does not blindly trust old mastery or assume unknown = beginner.

Phase 5 — Evaluator + mastery engine

Build: Add structured answer evaluation, deterministic state-update rules, and evidence history.

Exit criterion: Learner responses change mastery/confidence in a repeatable, testable way.

Phase 6 — Adaptive routing loop

Build: Add re-teach / practice / continue / stop conditional edges.

Exit criterion: The system performs multiple adaptive cycles without developer intervention.

Phase 7 — Learning-style inference

Build: Record behavioral signals and update the learner profile over time. Support domain-specific preference overrides.

Exit criterion: Two learners with the same knowledge state can receive meaningfully different explanations.

Phase 8 — Latency + streaming

Build: Stream tutor output, measure p50/p95 node latency, cap LLM/tool calls, and skip unnecessary nodes.

Exit criterion: Normal turns feel interactive rather than agentically slow.

Phase 9 — Android product integration

Build: Connect Compose UI, Room, session history, progress, and resilient sync behavior to the backend.

Exit criterion: The system becomes a real mobile product rather than a backend demo.

Phase 10 — LangSmith evaluation + portfolio polish

Build: Create datasets, traces, regression tests, latency/cost metrics, architecture diagrams, and README evidence.

Exit criterion: You can demonstrate why the system is reliable and explain tradeoffs in interviews.

13. How Phases Avoid Rewriting

Each phase should extend an interface or add a node rather than replace the previous architecture.

Phase 1:

Understand → Load fake state → Tutor

Phase 1B:

Understand → Load base state → Tool-enabled Tutor

- └─ get_mastery()
- └─ get_dependencies()
- └─ answer

Phase 2:

Understand → Load PostgreSQL state → Tool-enabled Tutor

Phase 3:

Understand → Tool-enabled Tutor → find_concept() → canonical concept

- └─ get_mastery()
- └─ get_dependencies()

Phase 4:

... → Check confidence

- └─ uncertain → Diagnostic
- └─ known → Tutor

Phase 6:

... → Tutor → Evaluate → Update → Route → loop

The node boundaries remain stable; implementations become richer. This is the key to learning incrementally without throwing away earlier work.

14. Deterministic vs LLM Responsibilities

Use LLM for	Use deterministic code for
Natural-language intent understanding	Authoritative database writes / transactions
Choosing among allow-listed read tools when extra context is needed	Mastery/confidence formulas and invariants
Teaching explanation and context-sensitive strategy	Thresholds and final routing guards
Open-ended answer evaluation	Objective quiz grading / validated state mutation
Generating diagnostics/exercises	Concept IDs, authorization, schema validation
Summarizing behavioral signals	Persistence, retries, idempotency
Selecting the intended concept from returned candidates using conversation context	Candidate search mechanics, canonical IDs, de-duplication, and persistence

15. Latency Budget Philosophy

Agentic does not mean slow. Keep the critical path small.

Normal turn:

Call #1 understand request

↓

Load minimal base state

↓

Call #2 tutor/orchestrator

├ optional find_concept / safe context tool call(s), max 1–2 rounds

└ stream personalized response

↓

return to user

Assessment turn:

+ evaluator call only when actual evidence needs semantic grading

- Cap graph loops and tool calls per turn.
- Do not invoke evaluator when there is no evidence to evaluate.
- Do not rebuild dependency graphs on every question.
- Load only the small slice of learner state relevant to the current concept.
- Use LangSmith to measure node-level and end-to-end latency.

16. V1 Non-Goals

- No RAG or vector database.
- No user-uploaded textbook grounding.
- No multi-agent supervisor hierarchy.
- No autonomous web browsing/research agent.

- No huge pre-generated knowledge graph.
- No custom fine-tuned model.
- No microservices, Kafka, Redis cluster, or Kubernetes.

17. V1 Success Criteria

- A user can ask to learn a concept and receive an explanation adapted to their learner state.
- Unknown/stale knowledge triggers a short diagnostic rather than false assumptions.
- User responses update mastery/confidence and change the next graph route.
- Learning-profile signals alter explanation strategy over repeated interactions.
- The agent can complete re-teach/practice/continue loops with bounded latency.
- State survives restarts and remains queryable in PostgreSQL.
- LangSmith traces show the exact nodes, decisions, latency, and structured outputs for a session.

18. Resume / Interview Framing

Avoid: “Built an AI tutor with LangChain.”

Better framing:

Built a stateful adaptive tutoring agent using LangGraph that maintains a persistent learner belief model, resolves sparse concept dependencies, diagnoses uncertain knowledge, dynamically selects teaching strategies, evaluates learner evidence with structured LLM outputs, and deterministically updates mastery and routing state.

The strongest interview discussion is not the prompt. It is why some nodes use LLM reasoning, why others are deterministic, how uncertainty is represented, how loops are bounded, and how you measured latency and evaluation quality.

19. First Implementation Task

Start with the smallest slice of the final architecture, not a throwaway prototype:

CURRENT PHASE 1 — keep this:

GraphState

↓

UnderstandRequestNode (LLM)

↓

LoadLearnerStateNode (service interface; hardcoded implementation first)

↓

TutorNode (LLM)

↓

END

NEXT — Phase 1B:

Make TutorNode tool-enabled

└─ get_learner_state()

└─ get_mastery()

```
└─ get_dependencies()
└─ get_recent_evidence()
(max 1–2 tool rounds)
```

LATER — Phase 3 concept resolution:

Tutor/Orchestrator may call `find_concept(query)`

↓

tool returns candidate concepts

↓

LLM selects intended concept using conversation context

↓

validated code persists only if a new concept is required

Once the current Phase 1 works, do not rewrite it. First add Phase 1B by giving the Tutor/Orchestrator node a small tool allow-list. Then replace hardcoded learner state with PostgreSQL behind the same interfaces. In Phase 3, add `find_concept(query)` so the LLM can resolve messy/ambiguous topic names through tool use while deterministic code owns canonical IDs and persistence. After that, add diagnostic, evaluator, update, and routing nodes one at a time.

V1 design principle: keep the system agentic where decisions and adaptation matter, deterministic where correctness and state integrity matter, and simple enough that every new phase extends—not replaces—the earlier work.